

Viz2D API Client Documentation

This client helps you create and track processing jobs on **Viz2D**.

Base URL: `https://api.viz2d.com`



Installation

```
npm install axios
```

Copy the API client into your project:

```
import api from "../viz2d-api";
```



Workflow Overview

Typical job flow:

1. Get a **job token** for a visualizer
 2. Create a **job** using the token and input file URL
 3. Poll the job status until **COMPLETED** or **FAILED**
 4. Read `outputUrl` when completed
-



Step 1 — Get Job Token

Each job must be created with a short-lived token tied to a visualizer.

Method

```
api.getJobToken(visualizerId: string)
```

Example

```
const res = await api.getJobToken("viz_123");  
const token = res.data;
```

Response

```
"eyJhbGciOiJIUzI1NiIsInR5cCI6I.."
```

Step 2 — Create Job

Create a processing job using the token and an input file URL.

Method

```
api.createJob(token: string, inputUrl: string)
```

Example

```
const jobRes = await api.createJob(token, "https://mycdn.com/  
const job = jobRes.data;
```



Request Body

```
{  
  "token": "string",  
  "inputUrl": "https://..."  
}
```

Response (Job Object)

```
{
  "id": "ckx9abc123",
  "status": "PENDING",
  "inputUrl": "https://mycdn.com/image.jpg",
  "outputUrl": null,
  "failReason": null,
  "createdAt": "2026-01-20T10:12:00.000Z",
  "updatedAt": "2026-01-20T10:12:00.000Z"
}
```

Step 3 — Get Job Status

Check job progress using the job ID.

Method

```
api.getJobById(jobId: string)
```

Example

```
const statusRes = await api.getJobById(job.id);
const jobStatus = statusRes.data.status;
```

Response

```
{
  "id": "ckx9abc123",
  "status": "COMPLETED",
  "inputUrl": "https://mycdn.com/image.jpg",
  "outputUrl": "https://cdn.viz2d.com/results/ckx9abc123.glb",
  "failReason": null,
  "createdAt": "...",

```

```
"updatedAt": "..."  
}
```

Job Status Values

Status	Meaning
PENDING	Job created, waiting to start
PROCESSING	Job is currently running
COMPLETED	Job finished successfully
FAILED	Job failed (see <code>failReason</code>)

Failure Handling

If status is `FAILED`, check:

```
job.failReason
```

Example:

```
{  
  "status": "FAILED",  
  "failReason": "Invalid input image format"  
}
```

Recommended Polling Strategy

Poll every **2–5 seconds** until terminal state.

Example helper:

```
async function waitForJob(jobId: string) {
  while (true) {
    const res = await api.getJobById(jobId);
    const job = res.data;

    if (job.status === "COMPLETED") return job;
    if (job.status === "FAILED") throw new Error(job.failReason);

    await new Promise(r => setTimeout(r, 3000));
  }
}
```

Authentication & Security

- Job creation requires a **token** generated for a visualizer.
- Tokens are short-lived and must not be reused across visualizers.
- Do not expose permanent credentials in frontend apps.

Recommended flow:

Client → Your Backend → Viz2D API

Let your backend fetch job tokens and create jobs if security is critical.

Job Data Model

```
id: string
status: "PENDING" | "PROCESSING" | "COMPLETED" | "FAILED"
inputUrl: string
outputUrl?: string
failReason?: string
createdAt: string (ISO date)
updatedAt: string (ISO date)
```

Limits & Best Practices

- Ensure `inputUrl` is publicly accessible
 - Avoid submitting duplicate jobs for same file
 - Implement retries only on network failures, not on FAILED jobs
 - Cache job results if users refresh pages
-

Full Example

```
import api from "../viz2d-api";

async function run() {
  const tokenRes = await api.getJobToken("viz_123");
  const token = tokenRes.data.token;

  const jobRes = await api.createJob(token, "https://mycdn.com");
  const jobId = jobRes.data.id;

  const finalJob = await waitForJob(jobId);
  console.log("Output:", finalJob.outputUrl);
}
```

If you want, I can also give you:

-  OpenAPI (Swagger) spec
-  Markdown docs formatted for website
-  Postman collection
-  Typesafe SDK version with Axios instances & interceptors